

CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications[☆]

Ruyue Xin^a, Peng Chen^{b,*}, Zhiming Zhao^{a,*}

^a Multiscale Networked Systems (MNS), University of Amsterdam, Amsterdam, The Netherlands

^b School of Computer and Software Engineering, Xihua University, Chengdu, China

ARTICLE INFO

Article history:

Received 15 July 2022

Received in revised form 19 March 2023

Accepted 19 April 2023

Available online 6 May 2023

Dataset link: https://github.com/AXinx/CausalRCA_code

Keywords:

Microservice applications

Root cause localization

Fine-grained

Causal inference

Monitoring data

ABSTRACT

Effectively localizing root causes of performance anomalies is crucial to enabling the rapid recovery and loss mitigation of microservice applications in the cloud. Depending on the granularity of the causes that can be localized, a service operator may take different actions, e.g., restarting or migrating services if only faulty services can be localized (namely, coarse-grained) or scaling resources if specific indicative metrics on the faulty service can be localized (namely, fine-grained). Prior research mainly focuses on coarse-grained faulty service localization, and there is now a growing interest in fine-grained root cause localization to identify faulty services and metrics. Causal inference (CI) based methods have gained popularity recently for root cause localization, but currently used CI methods have limitations, such as the linear causal relations assumption and strict data distribution requirements. To tackle these challenges, we propose a framework named CausalRCA to implement fine-grained, automated, and real-time root cause localization. The CausalRCA uses a gradient-based causal structure learning method to generate weighted causal graphs and a root cause inference method to localize root cause metrics. We conduct coarse- and fine-grained root cause localization to evaluate the localization performance of CausalRCA. Experimental results show that CausalRCA has significantly outperformed baseline methods in localization accuracy, e.g., the average AC@3 of the fine-grained root cause metric localization in the faulty service is 0.719, and the average increase is 10% compared with baseline methods. In addition, the average Avg@5 has improved by 9.43%. Codes and data are open-sourced and can be found in our Github repository [CausalRCA](https://github.com/AXinx/CausalRCA).

© 2023 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Microservices architecture (Balalaie et al., 2016) builds cloud applications by decomposing the system functionalities into multiple independently deployable units, making applications more resilient, robust, and adaptable to dynamic cloud environments. The performance of a microservice application is vital to guarantee the quality of user experience (QoE) and service (QoS) (Xin et al., 2023). However, performance anomalies, such as degraded response time, are inevitable due to the large scale and complex dependencies of services, causing enormous economic loss and user dissatisfaction (Li et al., 2022a). Furthermore, the performance of applications heavily depends on the underlying resources (Gregg, 2014); for example, high CPU usage results in a congested queue and growing latency (Ibidunmoye et al., 2015).

In order to enable application operations to take actions to resolve performance anomalies effectively, root cause localization to identify faulty services or resources is at the core of software maintenance for online service systems (Chen et al., 2020).

A microservice application can be observed by monitoring tools, which help operators to detect performance anomalies (Chen et al., 2022b; Song et al., 2023). However, performance anomaly detection only notifies operators when an anomaly occurs. To effectively handle performance anomalies, operators need to be informed about where the anomaly occurs (e.g., the faulty service) and what causes the anomaly (e.g., the memory leak). Root causes of a performance anomaly can be localized at different granularity: coarse-grained and fine-grained (Chen et al., 2014). Coarse-grained means that only faulty services can be identified, and the corresponding action will be the migration or restart of the entire service (Wu et al., 2021c), which is simple and straightforward but may not recover anomalies and has a higher risk of affecting other services and longer recovery times (Wu et al., 2020a). The developer of Instana Autotrace¹ emphasized

[☆] Editor: Ying Zou.

* Corresponding authors.

E-mail addresses: chenpeng@mail.xhu.edu.cn (P. Chen), z.zhao@uva.nl (Z. Zhao).

¹ <https://www.instana.com/>

the importance of identifying anomaly locations and root causes to avoid delays associated with restarting services, as this may not solve anomalies (Engelbert, 2020).

At a fine granularity, root cause localization will identify not only the faulty service but also the underlying resources through monitoring metrics of the service (Wu et al., 2021b). Operators can choose accurate actions to mitigate the performance anomaly using fine-grained root cause when pinpointing indicative metrics on the faulty service (Chen et al., 2014; Wu et al., 2020b). For example, service scale-out has a positive effect and shorter recovery time compared with service restart in the case of underlying resources being insufficient (Wu et al., 2020a). In industry, fine-grained root cause localization attracts much attention. The CCF AIOps Challenge,² jointly organized by industry and academia, aims to solve problems in real IT operations scenarios based on production systems of industrial companies (e.g., Sougo, eBay, Tencent) (Li et al., 2022b), providing the performance diagnosis challenge for microservice systems and requires localizing root causes at the metric level in 2020.³ Instana Autotracer,⁴ Google Cloud Operations⁵ are commercial platforms to identify root causes to help developers and operators fix performance issues (Vergadia, 2021). However, these platforms work with trace data that requires integrating tracking codes into applications and require time and expert technologies to analyze data. Monitoring data, which is different from trace data, can be readily collected and utilized for fine-grained root cause localization in microservice applications, aiding service operators in efficiently and cost-effectively identifying faulty services and pinpointing faulty metrics to resolve performance anomalies (Chen et al., 2014).

To track the root cause localization problem of microservice applications, some research has developed in recent years. We can classify them into log-based, trace-based, and metric-based according to the data sources (Soldani and Brogi, 2022). Log-based (Aggarwal et al., 2020) and trace-based (Kim et al., 2013; Weng et al., 2018) research have limitations, such as complex real-time processing and information extraction. On the other hand, metric-based research uses real-time monitoring data, including service latency and system resources, and focuses on localizing faulty services and resource metrics. This kind of research can assist anomaly recovery in taking actions like resource scaling easily without intervention of application source code (Soldani and Brogi, 2022). Nowadays, most metric-based research is coarse-grained (Lin et al., 2018; Guan et al., 2018; Wang et al., 2018; Ma et al., 2019, 2020a,b), and fine-grained root cause localization is starting to catch the attention of researchers (Meng et al., 2020; Wu et al., 2021a). As for localization methods in metric-based research, causal inference (CI) based methods that can model causal effects between services have been developing recently. For example, CausalInfer (Chen et al., 2016) applies the PC algorithm (named after its authors, Peter and Clark) (Spirtes et al., 2000), and MicroDiag (Wu et al., 2021a) uses the linear non-Gaussian acyclic model (LiNGAM) (Shimizu et al., 2011) to obtain causal graphs of metrics, which can be seen as anomaly propagation paths. However, currently used CI methods have limitations, such as uncertainty about some causal relations between metrics and strict assumptions about input data and causal relations (Wu et al., 2021b). Therefore, advanced CI methods can be considered for fine-grained root cause localization

to discover anomaly propagation paths and improve localization performance.

Fine-grained root cause localization in a microservice application is challenging because (1) services are often heterogeneous and have different characteristics, which may result in diverse anomaly symptoms for the same issue; (2) the complex dependency between microservices makes it difficult to model the anomaly propagation resulting from faulty services; (3) a large number of anomalous metrics introduced in a system makes it hard to find out the root one for a performance anomaly. To address these challenges, we formulate our main research question: *how to pinpoint the root cause of performance anomalies at a fine granularity based on monitoring data?* Three sub-questions are proposed: (1) *how to model anomaly propagation between monitoring metrics using CI methods?* (2) *How to precisely determine the root cause based on the propagation model?* (3) *How to evaluate the performance of root cause localization result?*

To answer the research question, we propose a CI-based fine-grained root cause localization framework named CausalRCA for microservice applications in this paper. The framework works when an anomaly is detected. Based on real-time monitoring data, CausalRCA will perform automatic root cause localization, including modeling anomaly propagation paths as a causal graph and ranking metrics to localize the root cause by traversing along the graph. Finally, CausalRCA outputs predicted root causes, which can be used by operators to determine strategies and recovery actions to solve the anomaly. In this paper, we evaluate the localization performance of CausalRCA on the sock-shop⁶ microservice benchmark. When a performance anomaly in sock-shop, such as the high response time of user requests, is detected, we can input monitoring metrics, including service latency and resource metrics of each service, to CausalRCA. After processing, the faulty service and root cause metric, for example, the memory usage metric in the order service, will be identified. Our experimental results show that CausalRCA has improved localization accuracy. For example, the average improvement of AC@5 for the fine-grained root cause metric localization in the faulty service is 9.43% compared with baseline methods.

Our contributions can be summarized below:

- We propose an automated, fine-grained root cause localization framework named CausalRCA, which analyzes monitoring data and localizes faulty services and system resources in real-time.
- We provide a gradient-based causal structure learning method in CausalRCA, which can automatically capture linear and non-linear causal relations between monitoring metrics.
- We conduct coarse- and fine-grained experiments to evaluate the localization performance of CausalRCA and demonstrate that the proposed framework has the best localization accuracy compared with baseline methods. For example, the average AC@3 is 0.719, which is a 10% improvement compared with baseline methods.

The rest of the paper is organized as follows. In Section 2, we review existing root cause localization research and CI methods. In Section 3, we propose a framework for root cause localization and a detailed introduction of each method. In Section 4, we design experiments from coarse-grained to fine-grained to evaluate the localization performance of our framework. Finally, discussion and conclusion are provided in Section 5 and Section 6.

² <http://iops.ai/>

³ <https://competition.aiops-challenge.com/home/competition/1484441527290765368>

⁴ <https://www.instana.com/>

⁵ <https://cloud.google.com/products/operations>

⁶ <https://github.com/microservices-demo/microservices-demo>

2. Related works

In recent years, research has developed for root cause localization in distributed system (Gholami and Srivastava, 2021), clouds (Weng et al., 2018; Wu et al., 2020b). Based on data sources, we can categorize these researches into three groups: log-based, trace-based, and metric-based (Soldani and Brogi, 2022). Log-based research (Aggarwal et al., 2020) mainly localizes root causes based on text logs parsing, which is hard to work in real time. Trace-based research (Kim et al., 2013; Weng et al., 2018) gathers information through the complete tracing of the execution paths and then identifies root causes along those paths. However, trace data only focuses on service level, and it is time-consuming for developers to understand source code well enough to extract trace information. In contrast, metrics-based research uses monitoring data collected from applications and underlying infrastructures to construct causal graphs and infer root causes. Metric-based research can achieve automated, real-time root cause localization based on multi-dimensional information. Therefore, this section will mainly review metrics-based research and CI methods.

2.1. Metric-based root cause localization research

Based on monitoring data, some researchers identify root causes of performance anomalies with statistical analysis, e.g., identifying anomalous monitoring metrics in parallel with detected anomalies. Wang et al. (2020) conduct correlation analysis based on mutual information to determine the root-cause metric for the anomalies they detect. However, given that correlation does not ensure causation (Calude and Longo, 2017), statistical analysis cannot pinpoint root causes. In addition, some researchers have developed topology graph-based analysis, which reconstructs the topology graph of a running application. For example, Wu et al. (2020b) generate a topology graph based on deployment information and extract a weighted anomalous subgraph by parsing resource-level monitoring data. Brandón et al. (2020) make snapshots of abnormal states of the application as graphs and then identify the root cause of a new anomaly by graph matching. This kind of research uses a reconstructed application topology graph to determine root causes, which can only be used for coarse-grained root cause localization.

To identify related papers for our research on CI-based root cause localization, we conduct a thorough search using Google Scholar. We use specific keywords such as “microservice”, “causal inference”, and “root cause localization” to narrow down the results, and limit the search to recent papers published between 2012 and 2022. We sort the papers based on relevancy and identified the two most related papers (Meng et al., 2020; Wu et al., 2021a). We carefully examine these papers and their benchmark methods and discover CauseInfer (Chen et al., 2014), which is a well-respected and influential research work. The CauseInfer focused on fine-grained real reasons causing performance problems, introducing a low-cost, black box cause inference system to build a causality graph and infer the causes from the graph. We use the snowballing technique (Wohlin, 2014) to explore CauseInfer’s citation papers to identify valuable papers and establish a comprehensive understanding of related research in CI-based root cause localization. We classify these research works into coarse- and fine-grained categories and summarize them chronologically, as shown in Table 1.

Existing CI-based root cause localization research works by constructing a causal graph based on monitoring data, i.e., including causal structure learning and root cause inference (Soldani and Brogi, 2022). Coarse-grained root cause localization usually builds a causal graph based on service level objective (SLO) metrics, such as service latency, and focuses on determining faulty

services. For example, Microscope (Lin et al., 2018; Guan et al., 2018) collect information on service interactions and monitoring service latency and then processes them based on PC and breadth-first search (BFS) algorithms to determine possible faulty service of detected anomalies. In addition, CloudRanger (Wang et al., 2018), MS-Rank (Ma et al., 2019, 2020a), and AutoMAP (Ma et al., 2020b) all exploit PC and random walk algorithms to build causal graphs and infer root causes. MS-Rank and AutoMAP use metrics not only service latency but also throughput, power, and resource consumption, whereas AutoMAP develops novel operations to refine the causal graph. Various coarse-grained root cause localization studies have been conducted, but they cannot assist operators in resolving application anomalies with accurate actions.

To address the drawback of coarse-grained root cause localization, some researchers focus on fine-grained root cause localization. Chen et al. first proposed CauseInfer (Chen et al., 2014, 2016), which infers the faulty service and root cause metric by constructing a causality graph of monitoring metrics in each service with the PC algorithm and traversing the metric causality graph with a depth-first search (DFS) method. After several years, Meng et al. provided MicroCause (Meng et al., 2020), which mainly focuses on the root cause metric localization in a faulty service. It provides a PC-based causal graph building method for time-series data and infers root causes with the random walk method. Afterward, Wu et al. proposed MicroDiag (Wu et al., 2021a), which focuses on fine-grained root cause localization and applies a DirectLiNGAM to build causal graphs and PageRank to infer root causes. Fine-grained root cause localization focuses mainly on SLO metrics and monitoring resource metrics of services and determining the root cause resource metric to help operators take actions like scaling resources (Tuli et al., 2022).

In Table 1, we can see that much research is focusing on coarse-grained root cause localization. However, fine-grained root cause localization was proposed early and has begun to attract the attention of more researchers in recent years. As for CI methods, we can see that causal structure learning methods like PC and LiNGAM are applied. At the same time, root cause inference methods, such as BFS and random walk, are popular. Based on these works, we consider precise root cause localization is more helpful for microservice application recovery from performance anomalies. At the same time, the localization accuracy of existing works can be improved; for example, the success rate of accurately identifying the root cause may be under 20% (Wang et al., 2018; Lin et al., 2018). Therefore, we are motivated to explore fine-grained root cause localization with advanced CI methods.

2.2. Causal inference methods

Causal inference methods, especially causal structure learning, have been researched for several years, and they play a vital role in many areas, such as genetics (Peters et al., 2017) and biology (Sachs et al., 2005). The causal structure learning problem can be formulated to learn a directed acyclic graph (DAG) from observational data. Methods can be classified into constrained-based, score-based, function-based, and gradient-based. Constrained-based methods, such as PC and FCI (Spirtes et al., 2000), use conditional independence tests to learn the skeleton of the casual graph and then orient the edges based on pre-defined orientation rules. Score-based methods, like GES (Chickering, 2002), assign scores to different causal graphs based on a pre-defined score function and then search over the space of DAGs to find the optimal one. Finally, function-based methods, like LiNGAM (Shimizu et al., 2006, 2011), construct a linear structural equation model (SEM) model based on linear and non-Gaussian assumptions, and solve it to get the DAG. These traditional methods contribute

Table 1
Classification of metric-based root cause localization research.

Reference	Year	Causal structure learning	Root cause inference	Input	Root cause	Granularity
Microscope (Lin et al., 2018; Guan et al., 2018)	2018	Parallelized PC	BFS	Service latency	Faulty service	Coarse-grained
CloudRanger (Wang et al., 2018)	2018	PC	Random walk	Service latency	Faulty service	Coarse-grained
MS-Rank (Ma et al., 2019, 2020a)	2019	PC	Random walk	Multi metrics	Faulty service	Coarse-grained
AutoMAP (Ma et al., 2020b)	2020	PC	Random walk	Multi metrics	Faulty service	Coarse-grained
CauseInfer (Chen et al., 2014, 2016)	2014, 2016	PC	BFS	Service latency	Faulty service	Fine-grained
MicroCause (Meng et al., 2020)	2021	PCTS	Random walk	Resource metrics	Root cause metric	Fine-grained
MicroDiag (Wu et al., 2021a)	2021	DirectLiNGAM	PageRank	Service and resource metrics	Root cause metric	Fine-grained

much to causal structure learning but have limitations. PC usually has ambiguous causal relations in causal graphs. GES takes a long time to match graphs, which makes it inappropriate to be applied to large-scale data. LiNGAM has strict linear and non-Gaussian assumptions, which makes it impractical.

With the development of deep neural networks, gradient-based methods are developed. Zheng et al. (2018) propose an equality constraint to the linear SEM, which enables a suite of continuous optimization techniques such as gradient descent. After that, Yu et al. (2019) provide a deep generative model and apply a variant of the structural constraint to learn the DAG. Gradient-based methods have no limitation of input data, can deal with linear and non-linear causal relations in data, and can automatically generate a weighted DAG. Gradient-based methods have been applied to medical (Upadhyaya et al., 2021) and biology (Demir et al., 2022). However, to the best of our knowledge, no research has applied gradient-based methods to root cause localization of microservice applications.

Based on DAGs generated by causal structure learning methods, researchers apply graph methods, like BFS (Beamer et al., 2012), random walk (Spitzer, 2001), and PageRank (Ridings and Shishigin, 2002), for root cause inference. The BFS is to traverse the graph and determine the abnormal node without descendants or with no abnormal descendants as a root cause. A random walk is walking through paths and choosing neighbors randomly in a graph. It determines the node most visited as the root cause. PageRank improves the random walk by adding the possibility of jumping to a random node, which will be used in this paper.

In conclusion, metric-based research can achieve automated and real-time root cause localization compared with log- and trace-based research. However, most existing research is about coarse-grained faulty service localization, while fine-grained root cause metric localization can be more helpful for rapid recovery and loss mitigation. CI-based methods are popular, but currently used methods have their limitations. Therefore, this paper will mainly focus on fine-grained root cause localization and explore gradient-based methods to build causal graphs.

3. Root cause localization framework

In this section, we propose a root cause localization framework named CausalRCA, including causal structure learning and root cause inference, and we will introduce detailed methods in the framework. All codes and data can be found in our Github repository [CausalRCA](https://github.com/AXinx/CausalRCA_code.git).⁷

3.1. Framework overview

The CausalRCA can automatically build anomaly propagation paths and localize root causes in real-time based on observable metrics. The CausalRCA framework consists of three components: monitoring metrics, causal structure learning, and root cause inference, as shown in Fig. 1.

The CausalRCA works when an anomaly occurs, such as the high latency of user requests, and it then automatically build anomaly propagation paths and localize root causes in real-time based on observable metrics. We first collect monitoring data, including service-level data, that is, service latency, and resource-level data, such as container CPU/memory usage. We use m_i^j to represent a monitoring metric in the service s_j , and all monitoring data is time-series data as shown in Fig. 1(a). Based on monitoring data, we then start the causal structure learning. The causal structure learning will automatically build a causal graph of metrics, which can be seen as anomaly propagation paths. We develop the causal structure learning with a gradient-based CI method, which can output a weighted DAG to represent causal relations between metrics as shown in Fig. 1(b). With the DAG, we start root cause inference to localize root causes. We apply PageRank to the weighted DAG and output a ranked list of all metrics, as shown in Fig. 1(c). Depending on the input data, the CausalRCA can be used for coarse- or fine-grained root cause localization. Coarse-grained works when input service latency, and CausalRCA will output the faulty service. Fine-grained works when input resource metrics, and CausalRCA will output the root cause metric. We evaluate the localization performance of CausalRCA in experiments in Section 4.

3.2. Causal structure learning

The causal structure learning component aims to build a causal graph of monitoring metrics. The causal graphs can be seen as anomaly propagation paths between metrics. We can use a DAG to represent the causal graph, in which each node represents a metric, and each edge represents a cause-effect relationship. Based on related work, we know that traditional causal structure learning methods have strict limitations on input data and relations. Therefore, we implement the causal structure learning in CausalRCA with a gradient-based method, DAG-GNN (Yu et al., 2019). DAG-GNN provides a deep generative model, which is a variational autoencoder (VAE) parameterized by a novel graph neural network (GNN) (Bruna et al., 2013), and applies a variant of the structural constraint to learn DAGs. Unlike other causal structure learning methods, the gradient-based method has no limitation of input data, can extract linear or non-linear causal relations between metrics, and automatically outputs a weighted DAG.

We use $X \in \mathbb{R}^{m \times n}$ (m is metrics, n is samples of each metric) to represent input data. To get a DAG from X , Zheng et al. (2018) adopt a linear SEM as a data generation model, which is $X = A^T X + Z$ ($A \in \mathbb{R}^{m \times m}$ is the weighted adjacency matrix. $Z \in \mathbb{R}^{m \times n}$ is the noise matrix). To ensure the acyclicity of the DAG, a constraint of A is proposed as:

$$h(A) = \text{tr}[(I + \alpha A \circ A)^m] - m = 0 \quad (1)$$

Based on the linear SEM, we can get $X = (I - A^T)^{-1}Z$, which can be written as $X = f_A(Z)$. This equation is a general form recognized as an abstraction of parameterized GNNs (Kipf and Welling, 2016). We can also see that X is generated from a latent

⁷ https://github.com/AXinx/CausalRCA_code.git

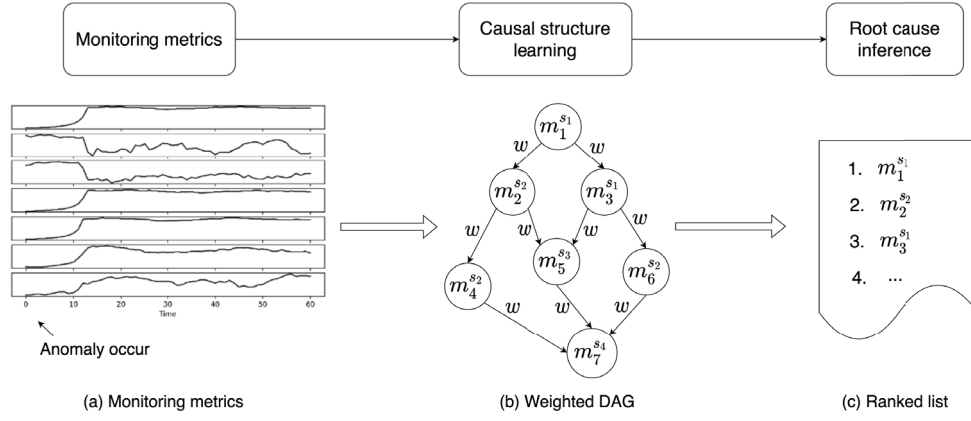


Fig. 1. CausalRCA: details of the root cause localization framework.

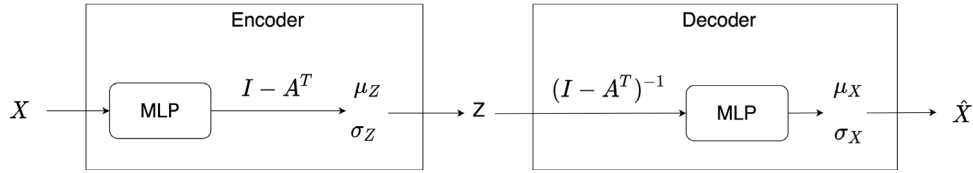


Fig. 2. Architecture of the causal structure learning method.

representation Z by defining a probabilistic graphical model. The generative model can be developed based on a VAE, and Z follows a standard Gaussian distribution (Kingma and Welling, 2013) as shown in Fig. 2.

With latent representation Z , we can define the decoder to reconstruct X as:

$$X = f_2((I - A^T)^{-1}f_1(Z)) \quad (2)$$

Then, the corresponding encoder can be defined as:

$$Z = f_4((I - A^T)f_3(X)) \quad (3)$$

Combining with deep neural networks, we use multilayer perceptron (MLP) to simulate f_1 , f_2 , f_3 , and f_4 , which all are parameterized functions. Based on VAE, the output of encoder and decoder are data distributions, so we get Z by sampling from μ_Z and σ_Z , and $\hat{X}$ by sampling from μ_X and σ_X .

For a VAE model, with a variational posterior $q(Z|X)$ to approximate the actual posterior $p(Z|X)$, evidence lower bound (ELBO) can be represented as:

$$\begin{aligned} L_{ELBO} &= E_{Z \sim q} [\log p(X|Z)] - KL(q(Z|X), p(Z)) \\ &= E_{Z \sim q} \left(-\frac{1}{2c} \|X - \hat{X}\|^2 \right) - KL(q(Z|X), p(Z)) \end{aligned} \quad (4)$$

Thus, the learning problem can be defined as:

$$\begin{aligned} \min_{A, \theta} f(A, \theta) &= -L_{ELBO} \\ \text{s.t. } h(A) &= 0 \end{aligned} \quad (5)$$

where θ is all the parameters of the VAE. For a nonlinear equality-constrained problem, we can use augmented Lagrangian method (Bertsekas, 1997) to solve it.

$$L_c(A, \theta, \lambda) = f(A, \theta) + \lambda h(A) + \frac{c}{2} |h(A)|^2 \quad (6)$$

where λ is the Lagrange multiplier and $c > 0$ is the penalty parameter. The following update rules are defined:

$$\begin{aligned} A^k, \lambda^k &= \arg \min_{A, \theta} L_c^k(A, \theta, \lambda^k) \\ \lambda^{k+1} &= \lambda^k + c^k h(A^k) \\ c^{k+1} &= \begin{cases} \eta c^k, & \text{if } |h(A^k)| > \gamma |h(A^{k-1})| \\ c^k, & \text{otherwise} \end{cases} \end{aligned} \quad (7)$$

In the augmented Lagrangian, the penalty parameter c is typically updated using an exponentially increasing function of the iteration number, and the Lagrange multiplier λ is correspondingly updated to converge to the optimal condition. The update rule for the penalty parameter c is important to balance the trade-off between feasibility and optimality in the optimization problem. The rule states that if the constraint violation at the next iteration is larger than the current violation, the value of c should be increased. Conversely, if the constraint violation at the next iteration is smaller than the current violation, the value of c should be kept the same. To achieve faster convergence and find optimal solutions, the update rule depends on two tuning parameters, η and γ . Usually, we set $\eta > 1$ to induce fast convergence and $\gamma < 1$ to limit the convergence speed (Yu et al., 2019). If γ is set too high, the convergence will be slow, while if η is set too high, the convergence will be fast, but the results may oscillate. Parameter analysis is provided in our experiments in Section 4.2.

During training, parameters A and θ will be updated every epoch. After training, we can get the A , which is the adjacency matrix of a DAG. For root cause localization in microservice applications, we define $X = [m_1^{s_1}, m_2^{s_1}, \dots, m_i^{s_j}, \dots]$. With this causal structure learning method, we can get a weighted DAG (G) which represent causal relations between metrics as shown in Fig. 1(b). Each node in G represent a metric, for example, $m_1^{s_1}$ means a metric in service s_1 . The edge from $m_1^{s_1}$ to $m_2^{s_1}$ indicates that a change in $m_1^{s_1}$ will result in a change in $m_2^{s_1}$ with the weight

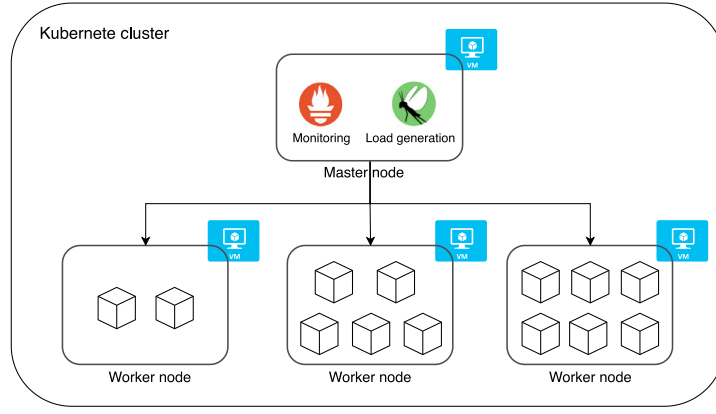


Fig. 3. The microservice application sock-shop deployed on VMs with Kubernetes.

w . Weight w represents the degree of the impact. If w is large, it indicates that a small change in $m_1^{s_1}$ will result in a large change in $m_2^{s_1}$. Furthermore, w can be either negative or positive, implying that an increase in $m_1^{s_1}$ may result in an increase or decrease in $m_2^{s_1}$. Based on the weighted DAG, we then use a root cause inference method to pinpoint the root cause metric.

3.3. Root cause inference

For the weighted DAG (G), we can rank metrics with the PageRank algorithm. PageRank works according to the number of incoming edges and the probability of anomalies spreading through the graph. We define P_{ij} as the transition probability of node i to j :

$$P_{ij} = \begin{cases} \frac{w_{ij}}{\sum_j w_{ij}}, & \text{if } w_{ij} \neq 0 \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

Here, w_{ij} is the weight between node i and j . We define P as the transition probability matrix. Then, we can get the PageRank vector v as proposed by Page et al. (1999) as:

$$v = \alpha P v + \frac{1 - \alpha}{n} \quad (9)$$

Here, n is the number of nodes, $\alpha \in (0, 1)$ is the teleportation probability, and it means that the random walk will continue with probability α and jump to a random node with probability $1 - \alpha$. We use the default setting $\alpha = 0.85$ (Becchetti and Castillo, 2006). To get results of the root cause inference method better, we first reverse edges in G and use the absolute value of all weights. After running the root cause inference method, we rank the PageRank scores of all nodes and get the ranked list as shown in Fig. 1(c). The higher the ranking on the list, the more likely the root cause is.

4. Experiments and results

To evaluate the root cause localization framework Causal-RCA, we conduct experiments on both coarse-grained and fine-grained. As for coarse-grained root cause localization, we design experiments to identify faulty services. As for fine-grained root cause localization, we first localize root cause metrics in the faulty service. In addition, taking into account the lack of understanding of services and underlying infrastructures of an application, we provide another fine-grained experiment to localize the root cause metric with all monitoring metrics in all services. In this section, we will introduce experimental settings and experimental results.

4.1. Experimental settings

4.1.1. Testbed

To evaluate our framework, sock-shop⁶, which simulates an e-commerce website that sells socks, is deployed. It is widely used as a microservice benchmark designed to aid demonstration and test microservices and cloud native technologies (Lin et al., 2018; Wu et al., 2020b, 2021b). Sock-shop consists of 13 services, which are implemented in heterogeneous technologies and communicate via REST over HTTP. Except for communication services, it contains 7 functional services, which are, *frontend* serves as the entry of user requests; *catalogue* provides product catalogue and information; *carts* holds shopping carts; *user* stores user accounts, including payment cards and addresses; *orders* place orders of login users from carts, and it consumes memory a lot; finally, *payment* and *shipping* services are provided for orders, which require network for processing transactions.

We deploy the sock-shop with Kubernetes on several VMs in the cloud, as shown in Fig. 3. In the Kubernetes cluster, we have one master node and three worker nodes. Their configurations are Ubuntu 18.04, 4vCPU, 16G RAM Memory, and 80G Disk. On the master node, we deploy open-source monitoring and visualization tools, Prometheus⁸ and Grafana⁹ respectively. Prometheus and Grafana are widely used for monitoring in microservice applications (Waseem et al., 2021; Ninio, 2022). Prometheus can keep monitoring the whole system and collecting both service-level and resource-level data (Wu et al., 2020b). In addition, a load generation tool, Locust¹⁰ is deployed on the master node to simulate workloads for the microservice application. On worker nodes, we deploy 13 services of the sock-shop application, and they are allocated to different VMs automatically by Kubernetes.

4.1.2. Anomaly injection

Microservice applications are deployed and distributed in clouds, and their performance is highly dependent on the resources of the underlying infrastructures. There are several common and widespread real performance anomalies in distributed systems (Mariani et al., 2018). Anomalous CPU consumption in VMs due to infinite loops, busy waits, or deadlocks of competing actions in applications can cause a slowdown of user requests (Sauvanaud et al., 2016). Memory leak, one of the most prominent software bugs that severely threaten the availability and security of systems (Kang et al., 2010), happen when

⁸ <https://github.com/prometheus/prometheus>

⁹ <https://github.com/grafana/grafana>

¹⁰ <https://locust.io/>

Table 2
Collected monitoring metrics.

Metrics type	Metrics
Service-level	Service latency
Resource-level	CPU usage
	Memory usage
	Disk read
	Disk write
	Network receive
	Network transmit

allocated chunks of memory are not freed after their use. Accumulations of unfreed memory may exhaust the system resource and lead to memory shortage and system failures. In addition, network resources are vulnerable to being attacked because of the frequent communication between servers and clients. Network latency anomalies usually originate from queuing or processing delays of packets on gateways (Sauvanaud et al., 2016). The three anomalies are common and frequent in microservice application (Wu et al., 2020b; Lin et al., 2018), which will be used to evaluate our framework.

Our method can be applied to any anomaly that manifests as increased microservice response time. In this evaluation, we inject the three common anomalies: CPU hog, memory leak, and network delay. We inject CPU hog by consuming CPU resources of each service. For memory leak, we allocate memory continuously for each service. For network delay, we enable traffic control to delay the network packets. We implement anomaly injection with the tool Pumba,¹¹ which can emulate network failures and stress-testing resources for Docker containers. Based on anomaly detection research (Jayatilaka et al., 2018; Huang et al., 2022), anomalies usually last several minutes, so each anomaly of each service we injected lasts 5 minutes, and the application will have 10 minutes to cold down before another injection.

4.1.3. Data collection

We deploy Prometheus to monitor the microservice application and collect monitoring data in real-time. Prometheus is configured to collect data every 5 seconds. We collect both service-level and resource-level data. At the service level, we collect the latency of each service. At the resource level, we collect container resource-related metrics, including CPU usage, memory usage, disk read and write, and network receive and transmit bytes, as shown in Table 2.

4.1.4. Baseline methods

Related work in Section 2 shows that CI-based root cause localization uses different causal structure learning methods. Our CausalRCA is developed based on a gradient-based causal structure learning method. Therefore, to evaluate the localization performance of our CausalRCA, we design baseline methods by combining different causal structure learning methods with PageRank. We chose the constraint-based method PC, the score-based method GES, and the function-based method LiNGAM.

For these baseline structure learning methods, we use their default parameter settings in causal-learn.¹² In CausalRCA, we use 2-layers MLP in the encoder and decoder, respectively. We set the learning rate as $1e-3$, and training epochs as 1000. In addition, we train the model with the Adam optimizer. We use $\eta = 10$ and $\gamma = 0.25$ as default in our experiments, which is proven to work well in Yu et al. (2019), and perform parameter analysis with $\eta = 100, 1000$ and $\gamma = 0.5, 0.75$. We run CausalRCA 10 times and take the average as the result of each experiment.

¹¹ <https://github.com/alexei-led/pumba>

¹² <https://github.com/cmu-phil/causal-learn>

4.1.5. Evaluation metrics

To evaluate localization accuracy, we use two performance metrics: $AC@k$ and $Avg@k$, which are the most commonly used metrics to evaluate rank results (Wu et al., 2020b). $AC@k$ represents the probability that the top k results given by a method include the real root cause. When the k is small, a higher $AC@k$ indicates that the method is more accurate in identifying the root cause. For each service and each anomaly type, $AC@k$ is calculated as follows:

$$AC@k = \frac{\sum_{i \leq k} R[i] \in V_{rc}}{\min(k, |V_{rc}|)} \quad (10)$$

where $R[i]$ is the result after ranking all metrics. V_{rc} is the root cause set. $Avg@k$ evaluates the overall performance of a method by computing the average $AC@k$, which is defined as:

$$Avg@k = \frac{1}{k} \sum_{1 \leq j \leq k} AC@j \quad (11)$$

We use $AC@1$, $AC@3$, and $Avg@5$ in our experiments. $AC@1$ evaluates if the top localized root cause is the real one, and it is the most restrictive and accurate metric. $AC@3$ is used to determine if the top three localized results have the real root cause. This metric is less accurate than $AC@1$, but it can still help operators quickly reduce root-cause candidates and localize the real ones. Finally, $Avg@5$ represents the average localization ability. The three metrics are commonly used in the root cause localization task, and they can fairly evaluate localization performance (Wu et al., 2020b; Meng et al., 2020).

4.1.6. Statistical testing

To assess the statistical significance of different RCA methods, we use the one-way analysis of variance (ANOVA) to test the difference between all RCA methods and the t-test to check the pairwise differences (Demšar, 2006). We use $Avg@5$ as the performance score of each RCA method. ANOVA is a hypothesis-testing framework for determining whether the between-group variation is significant. The F-statistic, calculated as the ratio of the between-group variation to the within-group variation, is used in ANOVA. The p -value associated with the F-statistic indicates the probability of obtaining an F-statistic as extreme as the observed one, assuming the null hypothesis is true. If the p -value is less than the significance level (usually 0.05), we reject the null hypothesis and conclude that there is a statistically significant difference among the RCA methods.

If the ANOVA test indicates a significant performance difference among these methods, we then use a t-test to determine the differences between each pair of RCA methods. The basic idea behind the t-test is to calculate a test statistic called the t -value, which measures the difference between the average performance scores of two methods relative to the variability within each method. Once the t -value has been calculated, it is compared to a critical value from a t -distribution. If the t -value is less than the critical value, we fail to reject the null hypothesis and conclude that there is insufficient evidence to suggest a performance difference between the two methods.

4.2. Experimental results

We provide the results of three experiments as below:

- Coarse-grained faulty service localization based on service latency of all services.
- Fine-grained root cause metric localization in the faulty service based on system-level metrics in the faulty service.
- Fine-grained root cause metric localization with all monitoring metrics in all services

We compare the localization performance of CausalRCA with baseline methods and explain the results.

Table 3

Localization accuracy of CI-based methods on localizing faulty services (Coarse-grained) for different anomalies.

Methods	PC-based	GES-based	LiNGAM-based	CausalRCA	Increase
CPU hog					
AC@1	0.1429	0.1429	0.1429	0.1873	4.44%
AC@3	0.2857	0.4286	0.7143	0.7175	0.32%
Avg@5	0.4286	0.4	0.5714	0.6244	5.3%
Memory leak					
AC@1	0	0	0.1429	0.2429	10%
AC@3	0.4286	0.1429	0.5714	0.6214	5%
Avg@5	0.4286	0.2286	0.5429	0.6143	7.14%
Network delay					
AC@1	0.1429	0	0.1429	0.1714	2.85%
AC@3	0.5714	0	0.4286	0.3857	–
Avg@5	0.4857	0.1714	0.4286	0.5057	2%
Average Avg@5	0.4476	0.2667	0.5143	0.5815	6.72%

4.2.1. Coarse-grained faulty service localization

We evaluate the performance of CausalRCA on localizing the faulty service that initiates performance anomalies. This localization is conducted based on service-level data, which is the latency of all services. Table 3 shows the localization accuracy compared with baseline methods for different anomalies. We can see that, when compared to baseline methods, CausalRCA has improved localization accuracy in terms of AC@1, AC@3, and Avg@5 in different anomalies by up to 10%. In addition, for CPU hog, causalRCA has the best performance in terms of AC@1, AC@3, and Avg@5. The AC@3 is 0.7175, which means that there is a 71.75% chance of finding the root cause in the top three metrics on the ranked list, which is slightly higher than the LiNGAM-based method. For memory leak, CausalRCA continues to outperform in terms of AC@1, AC@3, and Avg@5. There is a 62.14% possibility of localizing the root cause in the top 3 metrics. For network delay, AC@3 is not good enough, but AC@1 and Avg@5 are higher than baseline methods. In general, the average AC@1 of CausalRCA is 0.2, which means that there is an average 20% possibility that the top 1 metric on the ranked list can be identified as the root cause. The averages AC@3 and Avg@5 of CausalRCA for the three anomalies are 0.5749 and 0.5815, respectively. The increase of average Avg@5 is 6.72%, showing the improvement in localizing accuracy compared with baseline methods. We provide statistical testing to show the significant difference between these RCA methods. We obtained a p -value of 0.0003 using the ANOVA method first, showing a significant performance difference between the four RCA methods. We further utilized t-tests to compare the performance differences between each pair of methods, and the resulting p -values are shown in Fig. 4. We can see that CausalRCA has a significant difference from baseline methods, while PC-based and GES-based methods have no significant difference.

We analyze the impact of parameters γ and η in causal structure learning on the root cause localization performance of CausalRCA. We use $\gamma = 0.25$ and $\eta = 10$ as default, and also set $\gamma = 0.5, 0.75$, and $\eta = 100, 1000$. The results can be found in Fig. 5. We can see that $\gamma = 0.25, \eta = 10$ has the best performance in CPU hog and Network latency, and it also has the best average performance of different anomalies. In addition, $\gamma = 0.75, \eta = 10$ performs best for memory leak, because a high γ can prevent too fast convergence and find better solutions, but it usually takes more time. Furthermore, we can see that $\gamma = 0.25, \eta = 1000$ performs poorly on CPU hog anomaly, but relatively well on memory leak anomaly, suggesting that a high η can lead to more variance in localization results.

We then provide a detailed performance of CausalRCA on localizing faulty services with different anomalies in Fig. 6. For CPU

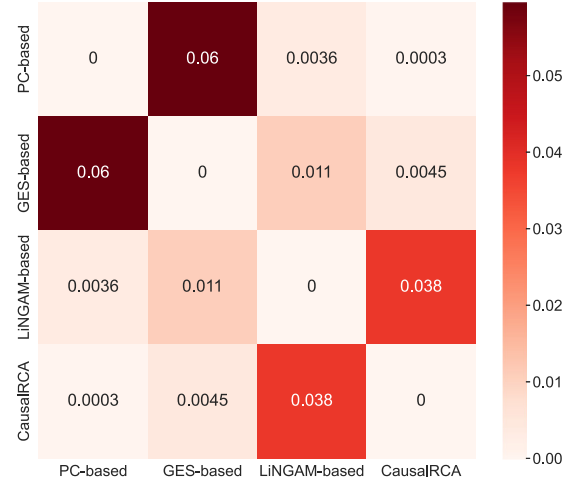
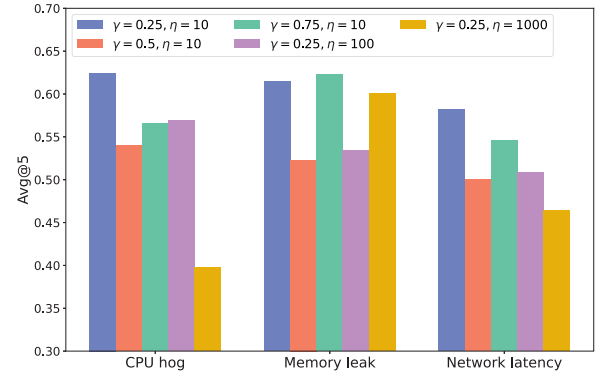
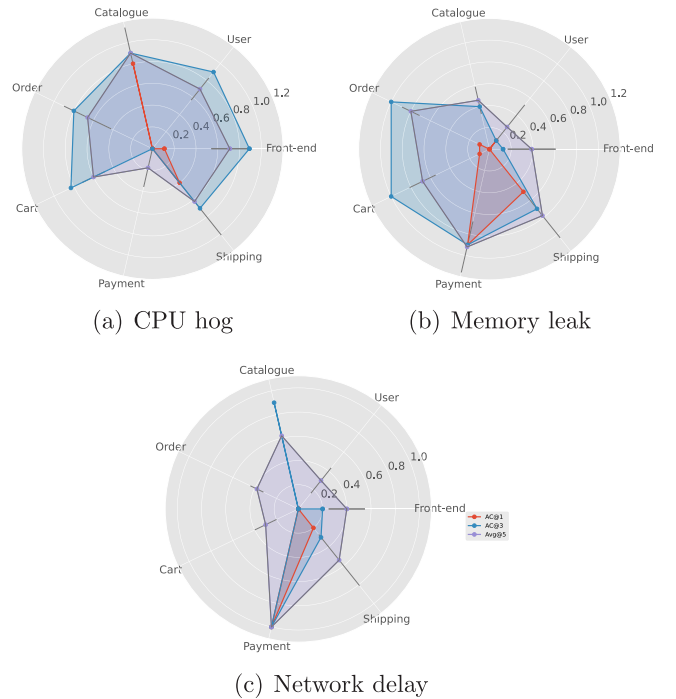
**Fig. 4.** P -value of RCA methods (Coarse-grained experiment)**Fig. 5.** Localization accuracy with different γ and η (Coarse-grained experiment)**Fig. 6.** Performance of CausalRCA on localizing faulty services with different anomalies.

Table 4

Localization accuracy of CI-based methods on localizing root cause metrics (Fine-grained) in faulty services for different anomalies.

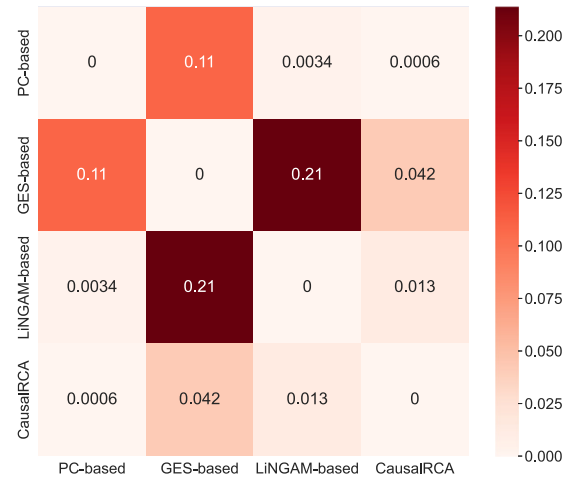
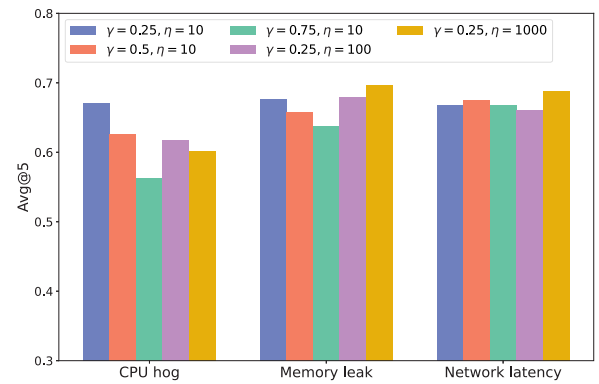
Methods	PC-based	GES-based	LiNGAM-based	CausalRCA	Increase
CPU hog					
AC@1	0.4286	0.1429	0	0.2286	–
AC@3	0.4286	0.5714	0.7143	0.7286	1.43%
Avg@5	0.4286	0.6	0.5714	0.67	7%
Memory leak					
AC@1	0	0	0.4286	0.2714	–
AC@3	0.1429	0.4286	0.5714	0.7143	14.29%
Avg@5	0.3429	0.4	0.6286	0.6771	4.85%
Network delay					
AC@1	0	0.1429	0.1429	0.2429	10%
AC@3	0.2857	0.5714	0.4286	0.7143	14.29%
Avg@5	0.2214	0.5143	0.5214	0.6571	13.57%
Average Avg@5	0.33	0.5048	0.5738	0.6681	9.43%

hog in Fig. 6(a), we can see that localization accuracy performs well on services except *payment*, because *payment* is not a CPU-intensive service. For the memory leak in Fig. 6(b), we can see that *front* – *end*, *user*, and *catalogue* perform worse than other services. The memory leak issue in these services does not affect their service latency much, making it difficult to identify cause-effect relations between services. In terms of network delay in Fig. 6(c), only service *payment* performs well, which explains the poor average localization performance of AC@3 in Table 3. *Payment* service relies heavily on the network, making it easy to localize the network delay issue. We plot the error bar for Avg@5 in Fig. 6 to represent the variation of our results, and we can see that the standard deviations of many results are not high.

4.2.2. Fine-grained root cause metric localization in the faulty service

Given the faulty service, we apply CausalRCA to container resource metrics and evaluate its performance on localization accuracy for different anomalies. Table 4 shows the localization accuracy of CausalRCA on localizing root cause metric in faulty service compared with baseline methods. For different anomalies, we can see that CausalRCA has improved localization accuracy compared with baseline methods in terms of AC@1, AC@3, and Avg@5 by up to 14.29%. In addition, for CPU hog, CausalRCA has the best performance in terms of AC@3 and Avg@5, while the AC@1 is worse than PC-based methods. For memory leak, the AC@1 of CausalRCA is worse than LiNGAM-based method, but it still has the best AC@3 and Avg@5. Finally, for network delay, CausalRCA outperforms in terms of AC@1, AC@3, and Avg@5. In general, the average AC@1 of CausalRCA is 0.2476, which means there is about a 25% possibility of determining the top 1 metric on the ranked list as the root cause. For the three anomalies, the average AC@3 is 0.719, which means there is a 71.9% possibility to localize the root cause metric in the top 3 metrics on the ranked list, and the average improvement is 10% compared with baseline methods. The average Avg@5 of CausalRCA is 0.6681, and the average increase is 9.43%. We consider the outperformance of CausalRCA is because resource metrics, such as CPU/memory usage, affect each other, which makes it easier to identify anomaly propagation with CI methods.

We also provide statistical testing to show the significant difference of these RCA methods. We first obtained a p -value of 0.0013 using the ANOVA method, showing a significant performance difference between the four RCA methods. The p -values obtained from t-tests are shown in Fig. 7. We can see that CausalRCA has a significant difference from baseline methods. In comparison, GES-based method has no significant difference with PC-based and LiNGAM-based methods.

**Fig. 7.** p -value of RCA methods (Fine-grained experiment)**Fig. 8.** Localization accuracy with different γ and η (Fine-grained experiment).

We evaluate the impact of parameters γ and η and present the findings in Fig. 8. The results indicate that $\gamma = 0.25$ and $\eta = 10$ perform the best in identifying CPU hog anomalies, while $\gamma = 0.25$ and $\eta = 100$ are most effective in detecting memory leak and network latency anomalies. In addition, $\gamma = 0.5$ and $\eta = 10$ outperform $\gamma = 0.5$ and $\eta = 100$ in detecting network latency anomalies and perform better than $\gamma = 0.75$ and $\eta = 10$ across all three types of anomalies. Overall, $\gamma = 0.25$ and $\eta = 10$ have the highest average localization performance. However, increasing γ and η could potentially lead to better solutions and improve localization accuracy.

We then provide the performance of CausalRCA on localizing root cause metrics in faulty services with the three anomalies in Fig. 9. We can see that AC@3 and AC@5 have consistent performance. For CPU hog, we can see that localization accuracy is low for *payment* and *shipping* services because they are insensitive to CPU resources. Because the memory leak issue manifests in multiple resource metrics, all services perform well for the memory leak. *Order* service has the best performance because it is highly related to memory usage. For network delay, we can see that *payment* and *shipping* have the best performance because they rely heavily on the network. We plot the error bar for Avg@5 in Fig. 9(b), and we can see there are some variances in the localization results, maybe caused by the dynamic nature of cloud environments or random fluctuation of resources in services, showing that the generality of CausalRCA can be explored more in the future.

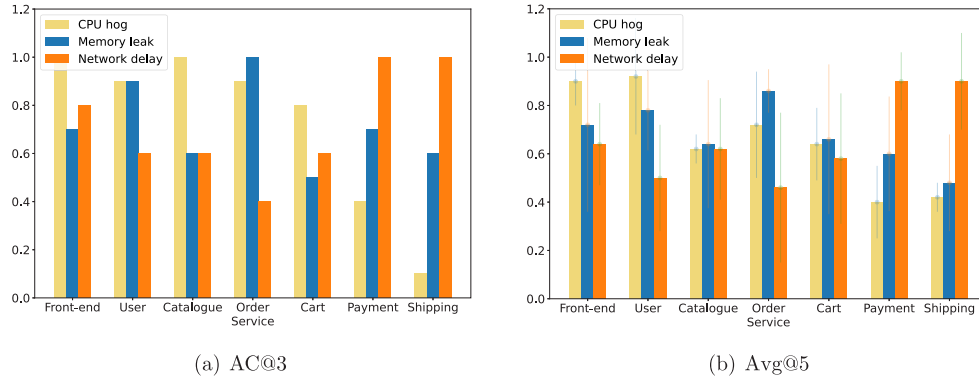


Fig. 9. Performance of CausalRCA on localizing root cause metrics in faulty services with different anomalies.

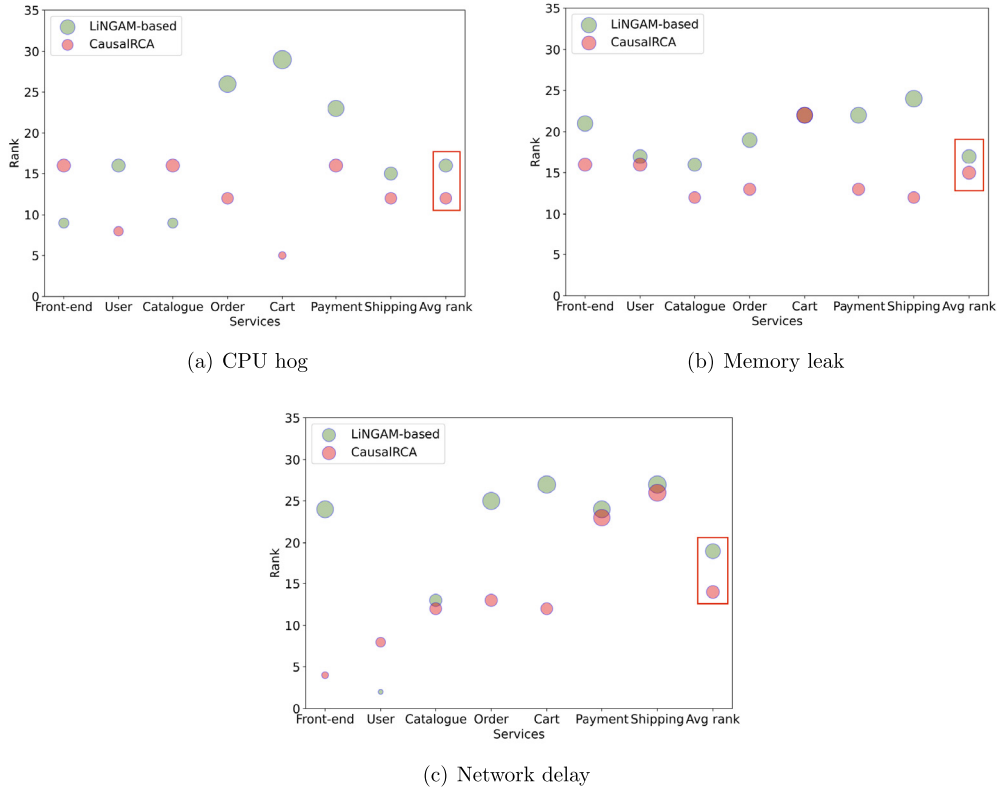


Fig. 10. Ranks of root cause metrics identified by CI-based methods.

4.2.3. Fine-grained root cause metric localization with all monitoring metrics

Considering that we are unknown to services and underlying infrastructures of an application, we conduct the fine-grained root cause localization with all monitoring metrics. We apply CausalRCA on all monitoring metrics to localize the root cause metric. We mainly show the ranks of comparison between the LiNGAM-based method and CausalRCA, because PC sometimes fails to extract causal relations between metrics, while GES takes too long to build a causal graph with too many nodes. For this fine-grained root cause localization, it is hard to identify the root cause metric in the top 1 or top 3 metrics, so we use the rank of root cause metrics to evaluate localization performance as shown in Fig. 10. We can see that the average rank of CausalRCA is about 13, which is lower than the LiNGAM-based method. The result shows that CausalRCA has better localization performance than the LiNGAM-based method. However, it is still challenging to extract causal relations between metrics and pinpoint the root

cause metric from multiple observable metrics. We apply the t-test to LiNGAM-based and CausalRCA methods and obtain the p -value of 0.0335, showing the significant difference between them. The impact of parameters γ and η is presented in Fig. 11. The results indicate that $\gamma = 0.25, \eta = 10$ performs the best in identifying CPU hog and network latency anomalies. For the memory leak anomaly, higher γ and η have better localization accuracy, and $\gamma = 0.75, \eta = 10$ is most effective. On average, the $\gamma = 0.25, \eta = 10$ has the best localization performance, while adjusting γ and η for the memory leak anomaly can improve localization accuracy effectively.

In conclusion, CausalRCA has a significant difference from baseline methods and better localization accuracy for coarse-grained and fine-grained root cause localization. In addition, we find that $\gamma = 0.25, \eta = 10$ in CausalRCA has the best localization performance on average. However, adjusting parameters can provide more potential for improving localization accuracy. Based on CI methods, we can see that anomaly propagation performs

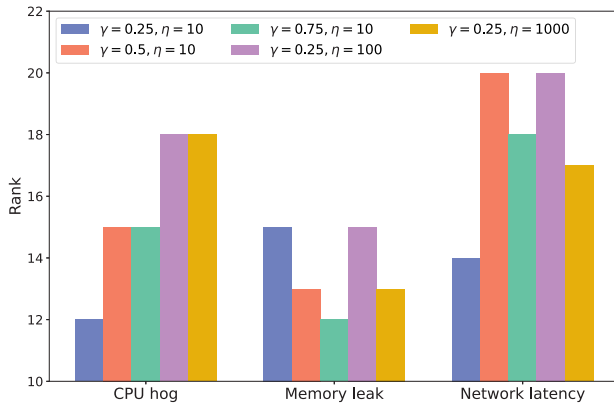


Fig. 11. Ranks of root cause metrics with different parameters γ and η .

differently in different services; for example, the *payment* service is sensitive to the network delay issue but nonsensitive to the CPU hog issue. As for fine-grained root cause localization, it is still challenging to pinpoint the root cause metric in all monitoring metrics. Therefore, it is more practical to consider the drill-down localization, i.e., identify the faulty service first and then determine the root cause metric in the faulty service.

4.3. Threats to validity

We analyze threats to our framework from the four categories: construct, internal, conclusion, and external validity based on Wohlin et al. (2012). The **construct threat** to validity mainly lies in the hyperparameters and evaluation metrics. We provide parameter analysis for two hyperparameters in CausalRCA, and results show that default parameters works well as provided in Yu et al. (2019) but tuning parameters carefully has the potential of improving localization accuracy. In addition, we use widely used evaluation metrics and provide statistical testing to evaluate the performance difference of different RCA methods.

The **internal threat** to validity mainly lies in the implementation of the framework, as errors or bugs in the implementation could affect the accuracy of the results. To reduce it, we have used established Python packages and conducted thorough testing. We have also repeated the experiments multiple times to ensure the reliability and consistency of our results. The **conclusion threat** to validity of our framework is related to the types of anomalies used in experiments. As microservice applications have a variety of performance anomalies that can affect the localization results (Mariani et al., 2018), we injected three different types of common and frequent anomalies to evaluate the effectiveness of our framework. We report and discuss the localization results for each individual anomaly type, and the experimental results demonstrate the superior performance of our framework on these anomalies.

The **external threat** relies on the configuration of microservice applications and the data collection strategies. In this paper, we investigate a specific configuration of a microservice application to evaluate the performance of CausalRCA, which may limit the generality of our framework. However, building complex infrastructures and repeating the experiments on multiple testbeds is extremely expensive, which is impractical for our experiments. In addition, the benchmark microservice application sock-shop is widely used in academia to aid the testing of microservices in clouds (Lin et al., 2018; Wu et al., 2020b), and it helps us mitigate this threat. On the other hand, the localization performance of our framework heavily relies on input data. To mitigate the threat, we adopt Prometheus, an open-source tool for real-time monitoring,

and collect service-level and resource-level metrics that present the status of a running microservice application. Currently, our framework performs well with anomalies injected over fixed time range anomalies, but the effect of different time ranges for CausalRCA can be explored more.

5. Discussion

This paper provides a framework called CausalRCA for root cause localization of microservice applications. The framework is developed with CI-based methods, including causal structure learning and root cause inference. We provide coarse-grained and fine-grained experiments to evaluate the localization performance of the framework. Our experimental results show that the framework has the best localization accuracy compared with baseline methods. However, some aspects can still be improved.

Our experiments show that CausalRCA performs well on localizing faulty services and root cause metrics in faulty services. However, localizing root cause metrics from all monitoring metrics is very hard. The average rank of root cause metrics is out of ten. We consider the improvement of localization accuracy can be researched more. First, data preprocessing, such as feature reduction, can be considered to reduce training time and improve localization accuracy. Next, we apply a gradient-based method to learn causal structures. The gradient-based method is applied to time-series monitoring data, which may ignore time lags in the original data. We consider that time lags in the data may help improve causal structure learning. For the root cause inference method PageRank, a personalized PageRank (Jeh and Widom, 2003), which considers the preferences of nodes, can be applied.

This paper mainly focuses on monitoring data to implement root cause localization. Monitoring data has multi-dimensional information and is easy to collect compared with trace and log data. However, trace data and log data contain accurate deployment information and service interactions, which can be used to calibrate the causal graph generated based on monitoring data. At the same time, the causal graph generated with CI methods can extract hidden relations between metrics. Therefore, we can consider combining different data resources to improve localization accuracy in the future.

This paper mainly focuses on improving localization accuracy of microservices, while efficiency is also important for achieving fast recovery. For the sock-shop benchmark application in this paper, we roughly estimate the time spent of our framework takes tens seconds, showing the cost of time may be lower than service migration (Chen et al., 2022a). However, for large-scale microservice, e.g., hundreds/thousands of services, the time cost for building the causality graph may be far greater than service migration. We will test the scalability and exact time cost of our framework and pay more attention to reducing training time in the future. For now, we mainly use the data collected in five minutes after the anomaly is detected. In the future, we will consider testing localization performance with different time ranges based on our CausalRCA.

6. Conclusion

This paper tackles the challenge of localizing root causes of performance anomalies in microservice applications. Root cause localization can be used to help operators achieve fast recovery of microservice applications. Therefore, it is important to guarantee localization accuracy at first. In addition, fine-grained root cause localization, which means identifying both the faulty service and resource related metrics in a faulty service, is necessary. With monitoring data, we provide a CI-based framework named

CausalRCA, which can automate localizing root causes with fine granularity and in real time.

The CausalRCA works with causal structure learning and root cause inference components. For causal structure learning, we propose a GNN-based method that uses a deep generative model and applies a variant of the structural constraint to learn the weighted DAG. Compared with other CI methods, the gradient-based method can extract non-linear causal relations between metrics. For root cause inference, we apply PageRank to visit the weighted DAG and return a ranked list of all metrics. We then provide experiments to evaluate the localization performance of CausalRCA.

In our experiments, we conduct three types of experiments: coarse-grained faulty service localization, fine-grained root cause metrics localization in faulty services, and fine-grained root cause metrics localization with all monitoring metrics. Our experimental results show that CausalRCA has better localization accuracy compared with baseline methods. Furthermore, based on CI methods, we can see that anomaly propagation performs differently between services, which gives operators a better understanding of microservice applications. In addition, it is difficult for fine-grained root cause localization with all monitoring metrics because anomalous metrics manifest diverse symptoms in different services. Therefore, fine-grained root cause localization with all monitoring metrics still needs to be improved. However, for microservice applications, we can still consider the drill-down localization, first identifying the faulty service, then pinpointing the root cause metric in the faulty service to identify fine-grained causes.

In the future, we will continue to improve the localization performance of CausalRCA. Hyperparameters tuning can be tested more in the future. The causal structure learning can consider time lag in monitoring data, and the root cause inference can be improved by adding the preferences of nodes. In addition, employing knowledge from trace and log data to calibrate the causal graph may improve localization accuracy and make the causal graph more reasonable. Finally, localization efficiency to achieve fast recovery needs to be tested and improved.

CRedit authorship contribution statement

Ruyue Xin: Conceptualization, Methodology, Writing – original draft. **Peng Chen:** Investigation, Writing – review & editing. **Zhiming Zhao:** Resources, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The link to the data and code is https://github.com/AXinx/CausalRCA_code.

Acknowledgments

This research work is funded by the EU Horizon 2020 and Horizon Europe research and innovation program under grant agreements 825134 (ARTICONF project), 862409 (BlueCloud project), 824068 (ENVRIFAIR project) and 101094227 (BlueCloud 2026). The work is also supported by LifeWatch ERIC and partially funded by the Science and Technology Program of Sichuan Province under Grant 2020JDR0067 and 2020YFG0326.

References

- Aggarwal, P., Gupta, A., Mohapatra, P., Nagar, S., Mandal, A., Wang, Q., Paraskar, A., 2020. Localization of operational faults in cloud applications by mining causal dependencies in logs using golden signals. In: International Conference on Service-Oriented Computing. Springer, pp. 137–149.
- Balalaie, A., Heydarnoori, A., Jamshidi, P., 2016. Microservices architecture enables devops: Migration to a cloud-native architecture. *IEEE Softw.* 33 (3), 42–52.
- Beamer, S., Asanovic, K., Patterson, D., 2012. Direction-optimizing breadth-first search. In: SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. IEEE, pp. 1–10.
- Becchetti, L., Castillo, C., 2006. The distribution of PageRank follows a power-law only for particular values of the damping factor. In: Proceedings of the 15th International Conference on World Wide Web. pp. 941–942.
- Bertsekas, D.P., 1997. Nonlinear programming. *J. Oper. Res. Soc.* 48 (3), 334.
- Brandón, Á., Solé, M., Huélamo, A., Solans, D., Pérez, M.S., Muntés-Mulero, V., 2020. Graph-based root cause analysis for service-oriented and microservice architectures. *J. Syst. Softw.* 159, 110432.
- Bruna, J., Zaremba, W., Szlam, A., LeCun, Y., 2013. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203*.
- Calude, C.S., Longo, G., 2017. The deluge of spurious correlations in big data. *Found. Sci.* 22 (3), 595–612.
- Chen, X., Bi, Y., Chen, X., Zhao, H., Cheng, N., Li, F., Cheng, W., 2022a. Dynamic service migration and request routing for microservice in multi-cell mobile edge computing. *IEEE Internet Things J.*
- Chen, Z., Kang, Y., Li, L., Zhang, X., Zhang, H., Xu, H., Zhou, Y., Yang, L., Sun, J., Xu, Z., et al., 2020. Towards intelligent incident management: why we need it and how we make it. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1487–1497.
- Chen, P., Liu, H., Xin, R., Carval, T., Zhao, J., Xia, Y., Zhao, Z., 2022b. Effectively detecting operational anomalies in large-scale IoT data infrastructures by using a gan-based predictive model. *Comput. J.* 65 (11), 2909–2925.
- Chen, P., Qi, Y., Hou, D., 2016. CauseInfer: Automated end-to-end performance diagnosis with hierarchical causality graph in cloud environment. *IEEE Trans. Serv. Comput.* 12 (2), 214–230.
- Chen, P., Qi, Y., Zheng, P., Hou, D., 2014. Causeinfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems. In: IEEE INFOCOM 2014-IEEE Conference on Computer Communications. IEEE, pp. 1887–1895.
- Chickering, D.M., 2002. Optimal structure identification with greedy search. *J. Mach. Learn. Res.* 3 (Nov), 507–554.
- Demir, A., Koike-Akino, T., Wang, Y., Erdoğan, D., 2022. EEG-gat: Graph attention networks for classification of electroencephalogram (EEG) signals. In: 2022 44th Annual International Conference of the IEEE Engineering in Medicine & Biology Society, EMBC, IEEE, pp. 30–35.
- Demšar, J., 2006. Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.* 7, 1–30.
- Engelbert, C., 2020. Rebooting your services for monitoring is so 2015. <https://www.instana.com/blog/rebooting-your-services-for-monitoring-is-so-2015/>.
- Gholami, A., Srivastava, A.K., 2021. Comparative analysis of ml techniques for data-driven anomaly detection, classification and localization in distribution system. In: 2020 52nd North American Power Symposium, NAPS, IEEE, pp. 1–6.
- Gregg, B., 2014. Systems Performance: Enterprise and the Cloud. Pearson Education.
- Guan, Z., Lin, J., Chen, P., 2018. On anomaly detection and root cause analysis of microservice systems. In: International Conference on Service-Oriented Computing. Springer, pp. 465–469.
- Huang, T., Chen, P., Li, R., 2022. A semi-supervised VAE based active anomaly detection framework in multivariate time series for online systems. In: Proceedings of the ACM Web Conference 2022. pp. 1797–1806.
- Ibidunmoye, O., Hernández-Rodríguez, F., Elmroth, E., 2015. Performance anomaly detection and bottleneck identification. *ACM Comput. Surv.* 48 (1), 1–35.
- Jayatilaka, H., Krintz, C., Wolski, R., 2018. Detecting performance anomalies in cloud platform applications. *IEEE Trans. Cloud Comput.* 8 (3), 764–777.
- Jeh, G., Widom, J., 2003. Scaling personalized web search. In: Proceedings of the 12th International Conference on World Wide Web. pp. 271–279.
- Kang, H., Chen, H., Jiang, G., 2010. PeerWatch: a fault detection and diagnosis tool for virtualized consolidation systems. In: Proceedings of the 7th International Conference on Autonomic Computing. pp. 119–128.
- Kim, M., Sumbaly, R., Shah, S., 2013. Root cause detection in a service-oriented architecture. *ACM SIGMETRICS Perform. Eval.* 41 (1), 93–104.
- Kingma, D.P., Welling, M., 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
- Kipf, T.N., Welling, M., 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.

- Li, Z., Zhao, N., Li, M., Lu, X., Wang, L., Chang, D., Nie, X., Cao, L., Zhang, W., Sui, K., et al., 2022a. Actionable and interpretable fault localization for recurring failures in online service systems. *arXiv preprint arXiv:2207.09021*.
- Li, Z., Zhao, N., Zhang, S., Sun, Y., Chen, P., Wen, X., Ma, M., Pei, D., 2022b. Constructing large-scale real-world benchmark datasets for AIOps. *arXiv preprint arXiv:2208.03938*.
- Lin, J., Chen, P., Zheng, Z., 2018. Microscope: Pinpoint performance issues with causal graphs in micro-service environments. In: *International Conference on Service-Oriented Computing*. Springer, pp. 3–20.
- Ma, M., Lin, W., Pan, D., Wang, P., 2019. Ms-rank: Multi-metric and self-adaptive root cause diagnosis for microservice applications. In: *2019 IEEE International Conference on Web Services. ICWS, IEEE*, pp. 60–67.
- Ma, M., Lin, W., Pan, D., Wang, P., 2020a. Self-adaptive root cause diagnosis for large-scale microservice architecture. *IEEE Trans. Serv. Comput.*
- Ma, M., Xu, J., Wang, Y., Chen, P., Zhang, Z., Wang, P., 2020b. Automap: Diagnose your microservice-based web applications automatically. In: *Proceedings of the Web Conference 2020*. pp. 246–258.
- Mariani, L., Monni, C., Pezzé, M., Riganelli, O., Xin, R., 2018. Localizing faults in cloud systems. In: *2018 IEEE 11th International Conference on Software Testing, Verification and Validation. ICST, IEEE*, pp. 262–273.
- Meng, Y., Zhang, S., Sun, Y., Zhang, R., Hu, Z., Zhang, Y., Jia, C., Wang, Z., Pei, D., 2020. Localizing failure root causes in a microservice through causality inference. In: *2020 IEEE/ACM 28th International Symposium on Quality of Service. IWQoS, IEEE*, pp. 1–10.
- Ninio, O., 2022. Practical guide on setting up prometheus and grafana for monitoring your microservices. <https://komodor.com/blog/setting-up-prometheus-and-grafana-for-monitoring-your-microservices/>.
- Page, L., Brin, S., Motwani, R., Winograd, T., 1999. The PageRank Citation Ranking: Bringing Order to the Web. *Tech. Rep.*, Stanford InfoLab.
- Peters, J., Janzing, D., Schölkopf, B., 2017. *Elements of Causal Inference: Foundations and Learning Algorithms*. The MIT Press.
- Ridings, C., Shishigin, M., 2002. Pagerank uncovered. Technical Paper for the Search Engine Optimization Online Community.
- Sachs, K., Perez, O., Pe'er, D., Lauffenburger, D.A., Nolan, G.P., 2005. Causal protein-signaling networks derived from multiparameter single-cell data. *Science* 308 (5721), 523–529.
- Sauvanaud, C., Lazri, K., Kaâniche, M., Kanoun, K., 2016. Anomaly detection and root cause localization in virtual network functions. In: *2016 IEEE 27th International Symposium on Software Reliability Engineering. ISSRE, IEEE*, pp. 196–206.
- Shimizu, S., Hoyer, P.O., Hyvärinen, A., Kerminen, A., Jordan, M., 2006. A linear non-Gaussian acyclic model for causal discovery. *J. Mach. Learn. Res.* 7 (10).
- Shimizu, S., Inazumi, T., Sogawa, Y., Hyvärinen, A., Kawahara, Y., Washio, T., Hoyer, P.O., Bollen, K., 2011. DirectLiNGAM: A direct method for learning a linear non-Gaussian structural equation model. *J. Mach. Learn. Res.* 12, 1225–1248.
- Soldani, J., Brogi, A., 2022. Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. *ACM Comput. Surv.* 55 (3), 1–39.
- Song, Y., Xin, R., Chen, P., Zhang, R., Chen, J., Zhao, Z., 2023. Identifying performance anomalies in fluctuating cloud environments: A robust correlative-GNN-based explainable approach. *Future Gener. Comput. Syst.* 145, 77–86.
- Spirtes, P., Glymour, C.N., Scheines, R., Heckerman, D., 2000. *Causation, Prediction, and Search*. MIT Press.
- Spitzer, F., 2001. *Principles of Random Walk*, Vol. 34. Springer Science & Business Media.
- Tuli, S., Casale, G., Jennings, N.R., 2022. Pregar: Preemptive migration prediction network for proactive fault-tolerant edge computing. In: *IEEE INFOCOM 2022-IEEE Conference on Computer Communications. IEEE*, pp. 670–679.
- Upadhyaya, P., Zhang, K., Li, C., Jiang, X., Kim, Y., 2021. Scalable causal structure learning: New opportunities in biomedicine. *arXiv preprint arXiv:2110.07785*.
- Vergadia, P., 2021. Introduction to google cloud's operations suite. <https://cloud.google.com/blog/topics/developers-practitioners/introduction-google-clouds-operations-suite/>.
- Wang, P., Xu, J., Ma, M., Lin, W., Pan, D., Wang, Y., Chen, P., 2018. Cloudranger: Root cause identification for cloud native systems. In: *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing. CCGRID, IEEE*, pp. 492–502.
- Wang, L., Zhao, N., Chen, J., Li, P., Zhang, W., Sui, K., 2020. Root-cause metric location for microservice systems via log anomaly detection. In: *2020 IEEE International Conference on Web Services. ICWS, IEEE*, pp. 142–150.
- Waseem, M., Liang, P., Shahin, M., Di Salle, A., Márquez, G., 2021. Design, monitoring, and testing of microservices systems: The practitioners' perspective. *J. Syst. Softw.* 182, 111061.
- Weng, J., Wang, J.H., Yang, J., Yang, Y., 2018. Root cause analysis of anomalies of multitier services in public clouds. *IEEE/ACM Trans. Netw.* 26 (4), 1646–1659.
- Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. pp. 1–10.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. *Experimentation in Software Engineering*. Springer Science & Business Media.
- Wu, L., Tordsson, J., Acker, A., Kao, O., 2020a. Microras: Automatic recovery in the absence of historical failure data for microservice systems. In: *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing. UCC, IEEE*, pp. 227–236.
- Wu, L., Tordsson, J., Bogatinovski, J., Elmroth, E., Kao, O., 2021a. MicroDiag: Fine-grained performance diagnosis for microservice systems. In: *2021 IEEE/ACM International Workshop on Cloud Intelligence. CloudIntelligence, IEEE*, pp. 31–36.
- Wu, L., Tordsson, J., Elmroth, E., Kao, O., 2020b. Microrca: Root cause localization of performance issues in microservices. In: *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium. IEEE*, pp. 1–9.
- Wu, L., Tordsson, J., Elmroth, E., Kao, O., 2021b. Causal inference techniques for microservice performance diagnosis: Evaluation and guiding recommendations. In: *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems. ACSOS, IEEE*, pp. 21–30.
- Wu, C., Zhao, N., Wang, L., Yang, X., Li, S., Zhang, M., Jin, X., Wen, X., Nie, X., Zhang, W., et al., 2021c. Identifying root-cause metrics for incident diagnosis in online service systems. In: *2021 IEEE 32nd International Symposium on Software Reliability Engineering. ISSRE, IEEE*, pp. 91–102.
- Xin, R., Liu, H., Chen, P., Zhao, Z., 2023. Robust and accurate performance anomaly detection and prediction for cloud applications: a novel ensemble learning-based framework. *J. Cloud Comput.* 12 (1), 1–16.
- Yu, Y., Chen, J., Gao, T., Yu, M., 2019. DAG-gnn: DAG structure learning with graph neural networks. In: *International Conference on Machine Learning. PMLR*, pp. 7154–7163.
- Zheng, X., Aragam, B., Ravikumar, P.K., Xing, E.P., 2018. Dags with no tears: Continuous optimization for structure learning. *Adv. Neural Inf. Process. Syst.* 31.